

OS Embarqués pour l'Edge Computing

Evan Galli, Élian Delmas et Eliot Menoret

Système d'alarme pour maison connectée sur
Arduino utilisant FreeRTOS et sur Raspberry Pi

<https://github.com/Other-Project/SI5-OS-EEC>

2026-01-18

Table des matières

I.	Introduction	1
II.	Architecture Matérielle et Logicielle	2
II.1.	Choix des composants et Rôles	2
II.2.	Instrumentation et Capteurs	2
III.	Interface de Communication	3
III.1.	Protocole	3
III.2.	Registres virtuels	4
IV.	Implémentation Logicielle	5
IV.1.	Gestion des tâches temps réel (Arduino & FreeRTOS)	5
IV.1.1.	Tâches	5
IV.1.2.	Synchronisation et Groupes d'Événements	6
IV.2.	Machine à États Distribuée	7
IV.3.	Application de Supervision	8
IV.3.1.	Modes d'exécution et Supervision	8
IV.3.2.	Gestion des Données et Persistance	8
IV.3.3.	Interface Utilisateur Physique (Écran LCD)	9
V.	Problèmes rencontrés	9
VI.	Conclusion	10
VI.1.	Analyse critique	10
VI.2.	Évolutions pertinentes	10
VI.3.	Répartition du travail	11
VI.4.	Utilisation de l'IA générative	11

I. Introduction

Dans l'architecture des systèmes de l'Internet des Objets (IoT), la tendance est au « Edge Computing », c'est-à-dire le traitement de la donnée au plus près du capteur. Plutôt que de centraliser toute la logique, on combine des microcontrôleurs capables d'interagir finement avec le réel, et des nano-ordinateurs qui disposent de la connectivité nécessaire pour agir comme passerelle vers le monde extérieur (réseau, interfaces utilisateur).

Le défi principal de cette architecture réside dans la répartition de la charge cognitive du système. Si la passerelle doit analyser elle-même les signaux bruts des capteurs, elle s'expose à deux risques : une surcharge de son bus de communication I²C et une complexité logicielle accrue pour filtrer les parasites ou gérer les timings électriques. Un système d'exploitation comme Linux n'est pas conçu pour scruter des changements d'états électriques rapides, mais pour gérer des flux de données. La problématique est donc : comment structurer le système pour que la passerelle ne manipule que des informations qualifiées, sans se soucier des contraintes physiques de l'acquisition ?

Ce projet répond à cette problématique par une architecture hiérarchisée où chaque composant joue un rôle spécialisé :

- Grâce à FreeRTOS, le microcontrôleur gère l'acquisition temps réel et le pré-traitement. Il encapsule la complexité matérielle pour n'exposer que des données métiers propres plutôt que des états de broches bruts.
- Libérée des tâches de bas niveau, la passerelle exécute un programme en Rust dédié à la supervision. Elle récupère les données métiers préparées via le bus I²C et gère les actions de haut niveau : journalisation, configuration et interface utilisateur.

Cette séparation transforme l'Arduino en un composant autonome qui offre une abstraction matérielle. Cela fiabilise le système : la détection est assurée par un OS temps réel sur microcontrôleur, tandis que la communication est gérée par la passerelle. Cette architecture rationalise les échanges sur le bus I²C : plutôt qu'un scrutage continu (« polling ») des états bruts des capteurs, la communication se limite à la transmission d'événements asynchrones qualifiés, libérant ainsi les ressources de la passerelle pour les tâches de supervision.

II. Architecture Matérielle et Logicielle

II.1. Choix des composants et Rôles

L'architecture retenue repose sur une approche distribuée hétérogène. Plutôt que d'utiliser une seule unité de calcul pour toutes les tâches, nous avons segmenté le système en deux entités distinctes reliées par un bus de communication. Ce choix permet d'attribuer à chaque composant le rôle qui correspond le mieux à ses caractéristiques matérielles intrinsèques.

- Un Arduino est utilisé comme microcontrôleur et agit en tant qu'unité d'acquisition intelligente, encapsulant les contraintes physiques pour n'exposer que des données qualifiées. L'intégration de FreeRTOS remplace la boucle séquentielle classique par un ordonnancement préemptif. Cela garantit le déterminisme des relevés capteurs tout en assurant une disponibilité constante du bus I²C pour la communication, sans blocage.
- Une Raspberry Pi 3B+ est utilisée en tant que nano-ordinateur. Contrairement au microcontrôleur, elle exécute un système d'exploitation complet (Linux), ce qui lui confère la puissance nécessaire pour gérer la logique de haut niveau, le système de fichiers et la pile réseau. Elle héberge l'application développée en Rust. Son rôle n'est pas de scruter les capteurs, ce qui consommerait des ressources inutilement, mais d'interroger périodiquement l'Arduino pour récupérer des événements déjà traités.

II.2. Instrumentation et Capteurs

L'interface avec l'environnement physique immédiat est assurée par un ensemble de périphériques branchés sur un *hat Base Shield V2* et pilotés par le microcontrôleur :

- L'authentification repose sur un module RFID (*Grove 125kHz RFID Reader*) pour l'identification des badges. Le traitement de ces données est délégué à la Raspberry Pi, qui centralise la base de données des badges et gère la validation nécessaires au désarmement du système.
- La surveillance d'intrusion est confiée à un capteur de distance à ultrasons (*Grove Ultrasonic Distance Sensor v2.0*). Celui-ci mesure en continu la distance face au capteur et l'augmentation au dessus d'un seuil est interprétée comme une intrusion (par ouverture de porte ou de fenêtre).
- Un bouton poussoir permet d'armer l'alarme, tandis qu'une diode électroluminescente (LED) et un buzzer fournissent respectivement un retour visuel de l'état (Armé/Désarmé) et un signal sonore dissuasif en cas d'alerte.

En tant que maître du système, la Raspberry Pi gère les périphériques destinés à l'interaction utilisateur :

- Un écran LCD (*Grove 16x2 LCD - Black on Yellow - v2.0*) fait office de terminal de contrôle, affichant l'état du système et permettant sa gestion (ajout et suppression de badges).
- Pour naviguer dans les menus affichés sur le LCD, la Pi lit, au travers d'un bus I²C avec un hat *Grove Pi+*, un potentiomètre (pour le défilement) et un bouton pour la validation.

III. Interface de Communication

III.1. Protocole

La communication entre la Raspberry Pi (Maître) et l'Arduino (Esclave) s'appuie sur un protocole applicatif personnalisé, encapsulé dans des trames I²C. Ce protocole permet la manipulation d'une carte de registres virtuelle gérée par l'Arduino.

Afin de prévenir toute corruption par du bruit électromagnétique, un mécanisme de validation par somme de contrôle (checksum) est mis en œuvre. Il correspond à une somme arithmétique simple des octets de données (charge utile). À la réception, le destinataire recalcule cette somme et rejette la trame en cas de discordance.

Pour envoyer une commande ou des données, la Raspberry Pi transmet une trame structurée contenant l'index du registre, le checksum de validation et la charge utile :

Trame = [REG_ID][CHECKSUM][DATA₀...DATA_n]

- REG_ID : Index du registre cible
- CHECKSUM : Somme des octets de données pour validation.
- DATA : Charge utile de longueur variable.

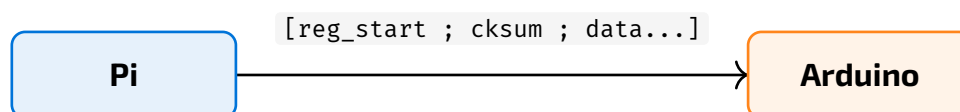


Fig. 1. – Opération d'écriture en I²C

La lecture est une opération plus complexe qui s'effectue en deux temps, car l'Arduino doit savoir à l'avance quelles données envoyer.

- La Pi effectue tout d'abord une écriture pour indiquer quel registre elle souhaite lire et le nombre d'octets attendus. Le bit de poids fort de l'adresse est forcé à 1 (par un masque 0x80) pour signaler qu'il s'agit d'une préparation à la lecture et non d'une écriture standard. Les registres étant peu nombreux, le 8ème bit est libre peut servir de drapeau.

Trame = [REG_ID | 0x80][COUNT]

- La Pi initie ensuite la lecture sur le bus. L'Arduino renvoie alors les données demandées suivies de leur checksum.

Trame = [DATA₀...DATA_n][CHECKSUM]



Fig. 2. – Opération de lecture en I²C

Ce protocole en deux étapes impose une synchronisation stricte : l'Arduino doit impérativement avoir traité la demande de préparation avant que la Raspberry Pi ne lance la lecture. Une latence trop élevée entraînerait la récupération de données invalides. Pour pallier ce risque, le code des interruptions de l'Arduino a maintenu aussi concis que possible afin de garantir la disponibilité de la lecture en quelques cycles d'horloge.

De plus, l'accès au bus I²C physique devant être strictement contrôlé pour éviter les conflits, nous avons encapsulé le driver I²C dans un Mutex.

III.2. Registres virtuels

La cartographie mémoire sert de contrat d'interface pour la communication I²C. Afin de garantir la cohérence des échanges, les définitions des registres sont dupliquées symétriquement : dans le fichier `consts.h` pour le firmware Arduino et dans le module `arduino_consts.rs` pour l'application Raspberry Pi.

Adresse(s)	Nom	Description
0x00	REG_STATUS	État du système (Désarmé, Armé, Triggered). Contrôle la machine à états.
0x01	REG_EVENTS	Drapeaux d'événements (Bitmask). Indique quels capteurs ont été activés.
0x02 .. 0x07	REG_RFID	Contient l'UID du dernier badge RFID détecté (sur 6 octets).
0x08	REG_ULTRASONIC	Seuil de distance pour la détection de mouvement.

Tableau 1. – Définition des registres virtuels mis à disposition en I²C

Le registre `0x00` pilote le comportement global du système. Lors d'une écriture sur ce registre par la Raspberry Pi, l'interruption déclenchée sur l'Arduino invoque la fonction de callback `onStatusChange`. Cette dernière a pour rôle d'activer ou de désactiver les tâches à exécuter par l'intermédiaire d'un `Event Group`.

Le registre `0x01` fonctionne comme un champ de bits. Cette méthode permet à l'Arduino de signaler plusieurs événements simultanés (ex: mouvement détecté pendant une lecture RFID) de manière compacte.

Les tâches FreeRTOS mettent à jour ce registre en temps réel. La structure du masque binaire est la suivante :

- Bit 0 (en LSB) : Bouton pressé
- Bit 1 : Ouverture détectée
- Bit 2 : Lecture d'un badge RFID
- Bits 3-7 : Inutilisés

Contrairement aux drapeaux d'événements binaires, l'identifiant unique (UID) d'un badge RFID nécessite plusieurs octets. Une plage de registres contigus, de `0x02` à `0x07` (6 octets), a été allouée pour stocker cette information.

Lorsqu'un badge est détecté, la tâche `vReadRfid` lit le code hexadécimal brut, le convertit en valeur numérique, puis décompose l'UID en une série d'octets pour les écrire séquentiellement dans ces registres.

Le registre `0x08` illustre la bidirectionnalité du protocole. Il permet au Raspberry Pi de configurer dynamiquement le seuil de détection du capteur ultrason, sans nécessiter de reprogrammation de l'Arduino. Cette valeur est quantifiée avec une résolution de 5 millimètres par pas.

IV. Implémentation Logicielle

IV.1. Gestion des tâches temps réel (Arduino & FreeRTOS)

L'utilisation de FreeRTOS sur le microcontrôleur ATmega328P permet de s'affranchir de la boucle infinie classique au profit d'une architecture multitâche préemptive. Cette approche garantit que le traitement des capteurs et la gestion des actionneurs sont effectués de manière déterministe, sans bloquer le bus de communication I2C.

IV.1.1. Tâches

L'application est structurée autour de quatre tâches principales, instanciées dans le main et gérées par l'ordonnanceur :

- Une tâche pour l'ultrason (`vUltrasonicTask`) qui mesure la distance périodiquement (toutes les 500 ms). Elle compare la valeur mesurée au seuil défini dans le registre `REG_ULTRASONIC_DISTANCE` pour détecter un mouvement. Si une intrusion

est confirmée, elle met à jour le registre d'état et déclenche l'événement de détection `EVENT_MOTION_DETECTED`.

- Une tâche pour le lecteur RFID (`vReadRfid`) exécutée toutes les 50 ms et qui effectue un « polling » du module RFID (« est-ce que des données sont à lire ? »). Lorsqu'un badge est présenté, l'UID de 48 bits est lu, découpé en octets et écrit dans les registres virtuels `REG_RFID` pour être accessible par la Raspberry Pi. Elle déclenche également le drapeau `EVENT_RFID_READ`.
- Une tâche pour le bouton (`vButtonTask`) qui surveille l'état du bouton poussoir physique pour permettre l'armement du système. Elle assure aussi la mise à jour du drapeau `EVENT_BTN_PRESSED` dans les registres partagés.
- Enfin, une tâche pour l'alerte (`vBlinkTask`) qui est responsable du retour utilisateur. Elle le clignotement de la LED et l'activation du buzzer, avec une périodicité de 500 ms.

IV.1.2. Synchronisation et Groupes d'Événements

Un point clé de l'implémentation est l'optimisation des ressources CPU grâce aux Event Groups de FreeRTOS. Plutôt que de vérifier continuellement des drapeaux d'état dans chaque tâche, le système utilise un objet de synchronisation global `xSystemStateGroup`.

En mode Désarmé, les tâches `vUltrasonicTask` et `vBlinkTask` sont en état bloqué, attendant respectivement les bits `BIT_ULTRASONIC_ENABLE` et `BIT_BLINK_ENABLE`. Elles ne consomment aucun temps processeur.

Lorsque le statut est modifié, la fonction `onStatusChange` se charge de lever ou effacer dans le groupe d'événements les bits correspondants.

Le passage en mode Armé réveille uniquement la tâche ultrason pour commencer la surveillance.

Le passage en mode Déclenché active les drapeau `BIT_ULTRASONIC_ENABLE` et `BIT_BLINK_ENABLE` ce qui permet de réveiller la tâche de clignotement (LED/Buzzer) pour signaler l'intrusion et désactive la tâche ultrason.

Cette architecture événementielle assure que le microcontrôleur consacre ses ressources uniquement aux fonctionnalités requises par l'état courant du système, maximisant ainsi la réactivité lors de la réception des interruptions I²C.

Enfin, la stabilité temporelle est assurée par l'utilisation de la fonction `vTaskDelayUntil`, qui garantit une fréquence d'exécution fixe des tâches, indépendamment de leur temps de traitement interne.

IV.2. Machine à États Distribuée

Le système évolue entre trois états :

- **Disarmed** (Veille): État de repos. Le capteur de mouvement est désactivé et les alertes sont éteintes.
- **Armed** (Surveillance): Activé par le bouton. Le système surveille le capteur ultrason et déclenche l'alerte en cas de détection. La LED est allumée de manière continue.
- **Triggered** (Alerte): Activé par une détection de mouvement. Le buzzer sonne et la LED clignote.

La machine à états est répartie entre les deux composants pour optimiser le fonctionnement du système : l'Arduino gère les transitions qui demandent de la réactivité, tandis que la Raspberry Pi gère celles qui nécessitent l'accès aux données (les badges).

Pour les changements d'état simples, l'Arduino agit seul via ses tâches FreeRTOS, sans attendre la Raspberry Pi :

- Armement (Désarmée → Armée) : L'appui sur le bouton est détecté localement par la tâche `vButtonTask`. Si le système est désarmé, l'Arduino passe directement le registre d'état à `STATUS_ARMED`. Ce qui active la tâche `vUltrasonicTask` de détection du mouvement au travers de la fonction `onStatusChange`.
- Déclenchement (Armée → Déclenchée) : Si la tâche `vUltrasonicTask` détecte un mouvement alors que l'alarme est armée, elle modifie elle-même l'état vers `STATUS_TRIGGERED`. Cela active immédiatement le buzzer et la LED.

Le retour à l'état « Désarmée » ne peut pas être décidé par l'Arduino seul, car il ne connaît pas la liste des badges autorisés.

- L'Arduino se contente de lire le numéro du badge RFID et de l'écrire dans les registres partagés.
- La Raspberry Pi lit ce numéro et vérifie s'il existe dans sa base de données SQLite.

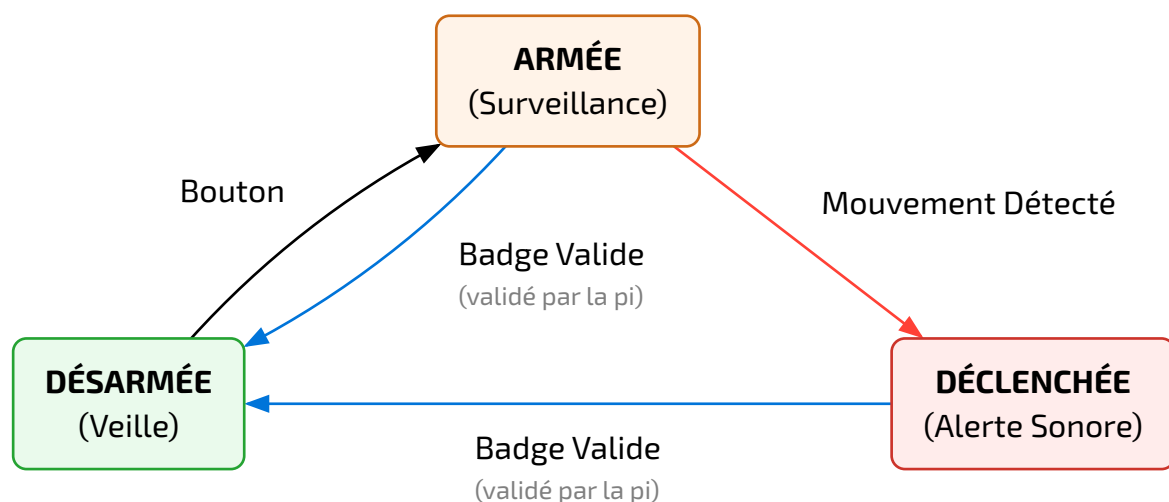


Fig. 3. – Machine à états finis de l'alarme

IV.3. Application de Supervision

L'intelligence de haut niveau du système est assurée par une application développée en Rust et hébergée sur la Raspberry Pi. Ce langage a été choisi principalement pour sa gestion stricte de la mémoire, ce qui permet d'éviter les erreurs de segmentation et assure une bonne stabilité à l'application de supervision.

L'application orchestre trois fonctions majeures : la supervision du système, la persistance des données et l'interaction directe avec l'utilisateur.

IV.3.1. Modes d'exécution et Supervision

L'architecture logicielle a été pensée pour s'adapter aussi bien à une utilisation en production qu'aux phases de maintenance.

Par défaut, le programme est configuré pour s'exécuter en mode « démon » (utilisable pour un service système d'arrière-plan), assurant une surveillance continue et silencieuse sans nécessiter d'interface graphique.

Pour les besoins d'administration avancée, l'application peut être lancée avec un argument spécifique (`-t` ou `--tui`) activant une TUI (Terminal User Interface). Cet outil de supervision est indispensable pour les opérations de gestion complexes inaccessibles via l'interface physique. Il offre une vue d'ensemble sur l'état courant de la machine à états et affiche en temps réel les journaux d'événements.

C'est spécifiquement au travers de cette interface que l'administrateur accède aux fonctionnalités étendues telle que l'ajustement de la distance seuil du capteur ultrason, de l'ajout de badges en leur associant un nom et une date d'expiration (pour les accès temporaires), ainsi que la possibilité de désactiver temporairement un badge sans le supprimer (le révoquer), ou de le supprimer définitivement via son nom. Il est aussi possible de consulter l'horodatage de la dernière utilisation pour chaque badge.

IV.3.2. Gestion des Données et Persistance

Afin de gérer les accès de manière pérenne, l'application intègre une base de données SQLite. Cette base locale permet de stocker et de structurer les informations relatives aux badges RFID.

Chaque badge RFID enregistré est défini par plusieurs attributs : un nom associé, une date d'expiration pour les accès temporaires, ainsi qu'un statut (activé ou désactivé) permettant de révoquer un accès sans supprimer définitivement l'entrée. Par ailleurs, le système assure une traçabilité en historisant l'horodatage de la dernière utilisation pour chaque badge.

C'est ce module central qui valide chaque scan de badge en interrogeant la base de données avant d'autoriser le désarmement de l'alarme.

IV.3.3. Interface Utilisateur Physique (Écran LCD)

Si la TUI demeure indispensable pour certaines actions telles que la gestion des badges temporaires, l'interaction quotidienne a été conçue pour être autonome via les périphériques matériels. L'écran LCD assure le retour d'information principal en affichant l'état de l'alarme.

Pour les opérations courantes, telles que l'ajout ou la suppression standard de badges, l'utilisateur dispose d'un menu embarqué accessible via le bouton. La navigation y est intuitive : le potentiomètre permet le défilement des options tandis que le bouton assure la validation. Enfin, un mécanisme de temporisation quitte automatiquement le menu après 10 secondes d'inactivité.

V. Problèmes rencontrés

Le premier défi a concerné la gestion de la mémoire sur l'Arduino. Bien que notre implémentation n'utilise pas d'allocation dynamique, évitant ainsi les problèmes d'instabilité ou de fragmentation, nous avons été confrontés à une limitation d'espace disponible. La configuration par défaut de FreeRTOS allouait une taille de tas (heap) trop importante par rapport aux capacités de l'ATmega328P, restreignant l'espace pour notre code. Nous avons donc réduit manuellement la configuration de FreeRTOS pour réduire l'emprunte mémoire. Parallèlement, l'intégration matérielle sur la Raspberry Pi a nécessité le remplacement du shield GrovePi initial, incompatible avec notre version de carte, par un modèle GrovePi+. Sur ce dernier, nous avons constaté que si l'écriture sur les ports digitaux était parfaitement fonctionnelle, la lecture y était impossible (sans que nous puissions l'expliquer). Pour contourner cette défaillance, nous avons déplacé le bouton de navigation sur un port analogique et traité le signal par seuillage logiciel.

La fiabilité de la communication inter-système a constitué la difficulté technique centrale du projet. La liaison I²C subissait des réceptions intermittentes de données corrompues, ce qui nous a conduits à implémenter une validation systématique par somme de contrôle (checksum). Plus critique encore, nous avons identifié une condition de concurrence (race condition) lors des lectures. Le problème survenait car la Raspberry Pi initiait la lecture des données avant que l'Arduino n'ait terminé de traiter la demande de préparation correspondante. Pour garantir la validité des échanges, nous avons réécrit le code des routines d'interruption (ISR) de l'Arduino. Cette optimisation visait à accélérer leur exécution afin de réduire le chevauchement temporel entre les requêtes d'écriture standard dans un registre et celles destinées à déclarer la zone de lecture.

Enfin, la mise au point du système a mis en lumière les contraintes inhérentes au développement sur microcontrôleur. Le débogage s'est avéré plus complexe que pour un programme standard, en raison de l'absence des outils d'analyse avancés disponibles sur un système d'exploitation complet.

VI. Conclusion

VI.1. Analyse critique

Notre solution a permis de valider la pertinence d'une architecture distribuée pour l'IoT. La séparation des tâches entre le Raspberry Pi et l'Arduino a parfaitement rempli son rôle : la délégation de la gestion matérielle à FreeRTOS a assuré une détection fiable et réactive des capteurs, déchargeant le Raspberry Pi des contraintes temps réel. L'utilisation de Rust sur la passerelle a également offert une robustesse appréciable, garantissant la stabilité du processus de supervision sans les erreurs de mémoire courantes en C/C++. Le protocole de communication I²C personnalisé, bien que complexe à mettre au point, s'est révélé robuste grâce à l'implémentation des checksums et des mécanismes de synchronisation (Mutex), éliminant efficacement les erreurs de transmission et les races conditions.

Cependant, le système présente certaines limitations architecturales. La dépendance critique envers le Raspberry Pi pour la validation des badges constitue un point unique de défaillance : si la passerelle ou le bus I²C dysfonctionne, l'alarme ne peut plus être désarmée, même avec un badge valide. De plus, l'interface physique actuelle manque de sécurisation, permettant à quiconque d'accéder au menu de configuration via le bouton sans authentification préalable.

VI.2. Évolutions pertinentes

Bien que l'architecture distribuée actuelle démontre la pertinence du couplage Raspberry Pi/Arduino, plusieurs évolutions permettraient de renforcer l'autonomie et l'ergonomie du système :

- Une synchronisation périodique des identifiants des badges valides directement dans la mémoire non-volatile (EEPROM) de l'Arduino permettrait de décentraliser la décision d'authentification. Cette modification rendrait le microcontrôleur capable de valider un désarmement de manière autonome, garantissant ainsi la continuité du service même en cas de rupture de la communication I²C ou de défaillance logicielle de la passerelle.
- Les fonctionnalités accessibles via l'écran LCD mériteraient d'être étendues.
- L'accès au menu de configuration physique, actuellement libre via le bouton poussoir, devrait être sécurisé pour prévenir toute modification malveillante des paramètres. L'implémentation d'un mécanisme de verrouillage, requérant le scan d'un badge autorisé renforcerait considérablement la sécurité physique du dispositif.
- L'introduction d'une temporisation d'entrée est nécessaire pour adapter le système aux contraintes réelles d'installation. Actuellement, la détection de mouvement déclenche une alerte immédiate. L'ajout d'un état intermédiaire de « pré-alarme » offrirait un délai configurable à l'utilisateur légitime pour atteindre le lecteur RFID et s'authentifier avant l'activation sonore du buzzer.
- La TUI pourrait être découplée du daemon et utiliser un protocole réseau pour échanger avec celui-ci.

VI.3. Répartition du travail

Le travail a été mené de manière collaborative par l'ensemble du groupe, notamment en ce qui concerne le développement Arduino et la rédaction de ce rapport, auxquels les trois membres ont contribué. Evan a cependant pris en charge plus spécifiquement le développement de l'application de supervision en Rust sur la Raspberry Pi, assurant ainsi l'intégration de la logique haut niveau.

VI.4. Utilisation de l'IA générative

Dans le cadre de ce projet, nous avons eu recours à des outils d'intelligence artificielle générative comme support de développement. Ces outils ont été utilisés pour accélérer l'écriture de portions de code standard et aider au débogage. Ils ont aussi été utilisés pour améliorer la qualité rédactionnelle du rapport. La conception de l'architecture, la logique métier et la validation finale du système restent le fruit de notre propre travail.